

ENCODER-ONLY TRANSFORMER

ABHISEK MOHAPATRA- AI25BTECH11001

I. ABSTRACT

In this report, we will try to predict ranks of a sequence of integers (place of the integer in ascending order). Well, it is not very hard to solve the problem but we want to see how the transformer encodes the order through attention and how it beats other methods such as MLP, RNN and LSTM. All the model and its architecture will be coded from scratch (however, using pytorch primitives to get a boost in training time).

II. BASELINES

We will go from MLP to LSTM, and see how they predict the ranks.

A. MLP

Well, MLP architecture is simple and straight forward.

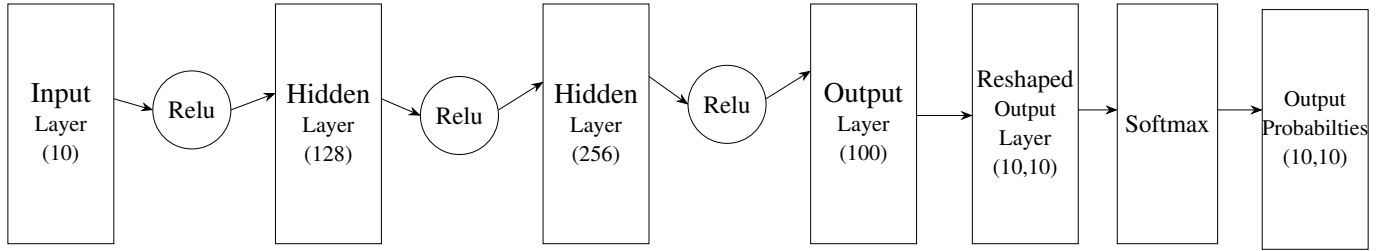


Fig. 1: MLP Architecture (the number denotes the node in each layer)

Here the input layer will be provided with the ten numbers whose rank to be predicted. And after that it predicts 100 values in the output layer which are then softmaxed to get the probabilities of each rank for the given number.

1) *Normalisation and Data preparation:* The input are normalised z-score of the original numbers. The output is directly fed into the loss function as `nn.CrossEntropy` can accept classes as input.

2) *Training:* The training dataset, validation dataset and test dataset are divided in ratio of 8:1:1. The training dataset then divided into batches of size 64. The optimiser used is `torch.optim.AdamW` ($learning_rate = 0.003$) and it is trained for 100 epochs.

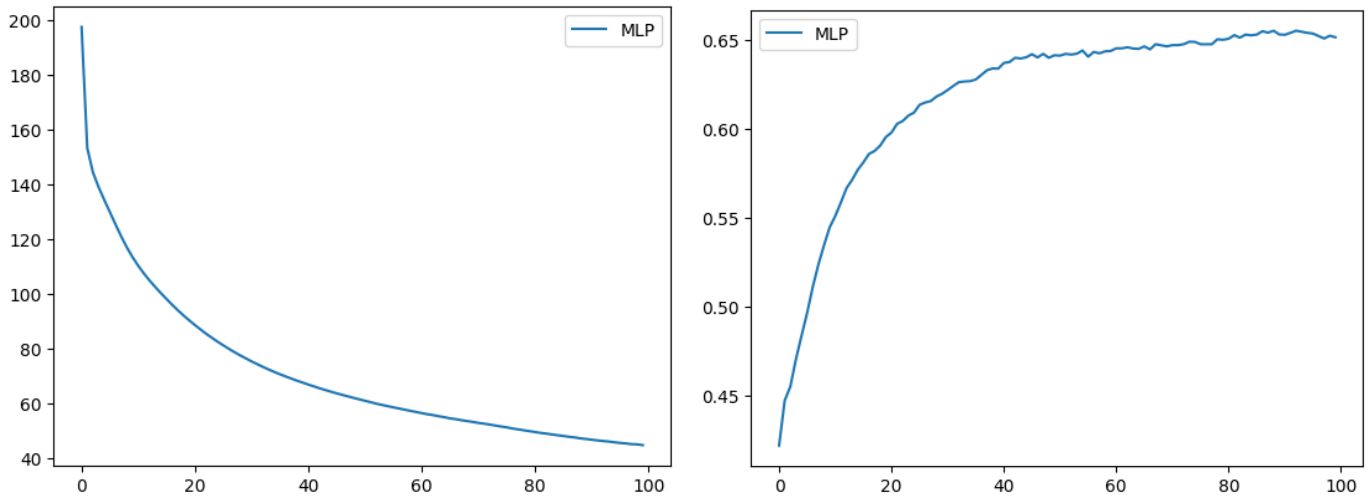


Fig. 2: (Left)loss over the epochs (Right) validation accuracy over epochs

As you can see, it saturates over 65% validation accuracy. Well, there are few more metrics apart from this.

3) Evaluation:

- 1) Test Accuracy (%): tensor(66.2100, device='cuda:0')
- 2) Full sequence correct (%): tensor(1.5000, device='cuda:0')
- 3) Atmost 1 incorrect sequence correct (%): tensor(10.1000, device='cuda:0')
- 4) Atmost 2 incorrect sequence correct (%): tensor(29.4000, device='cuda:0')
- 5) Average no of rank predicted incorrect in a sequence : tensor(3.3790, device='cuda:0')

A few testcases are here:

- 1) 5 testcase x : [0.610968 1.1796409 0.51500446 -0.32734233 0.6749437 -1.1092676 0.26621005 -1.7490246 0.9806054 -1.0417378]
 pred : [5 9 5 3 6 2 4 0 7 2]
 true : [6 9 5 3 7 1 4 0 8 2]
- 2) 10 testcase
 x : [0.7435295 0.95780313 -1.2706426 -1.9134635 0.6399639 -0.63853544 -0.3314099 0.9292333 0.63282144 0.25070015]
 pred : [8 8 1 0 5 2 3 9 6 4]
 true : [7 9 1 0 6 2 3 8 5 4]
- 3) 15 testcase
 x : [0.01127135 0.3719545 -1.2436054 0.3944972 0. 0.9317648 -0.2329412 -2.103985 0.7626946 1.1083492]
 pred : [3 6 1 6 4 9 2 0 7 9]
 true : [4 5 1 6 3 8 2 0 7 9]
- 4) 20 testcase
 x : [-0.35796353 -1.284492 1.050948 -0.5256211 0.8803491 0.38325924 -0.7168095 1.0627134 -1.5021527 1.009769]
 pred : [3 1 9 4 7 5 2 8 0 7]
 true : [4 1 8 3 6 5 2 9 0 7]
- 5) 25 testcase
 x : [1.5437944 0.6811236 1.2894996 -1.034561 -0.93155557 -0.78026634 0.46545592 0.28841528 -1.2534477 -0.26845798]
 pred : [9 6 8 1 2 2 6 5 0 4]
 true : [9 7 8 1 2 3 6 5 0 4]

- 6) 30 testcase
 x : [-1.0927486 0.2008844 0.7167743 -1.0614825 0.24387524 0.9434532 0.19697616 -1.6828954 0.01719633 1.517967]
 pred : [1 4 7 2 5 8 6 0 3 9]
 true : [1 5 7 2 6 8 4 0 3 9]
- 7) 35 testcase
 x : [0.42147765 0.28269842 1.2644329 -1.3312526 -0.8275354 0.24671862 -0.71959597 0.1439192 -1.1462135 1.6653507]
 pred : [6 5 8 0 2 5 2 5 1 9]
 true : [7 6 8 0 2 5 3 4 1 9]
- 8) 40 testcase
 x : [0.8400531 -1.0256346 0.82031035 0.2576426 0.92889535 -0.8430145 -0.76897925 -0.4580313 -1.3069686 1.555727]
 pred : [7 2 6 5 7 3 3 3 0 9]
 true : [7 1 6 5 8 2 3 4 0 9]
- 9) 45 testcase
 x : [1.405338 -0.5287208 -0.62235147 1.0922605 -0.06056734 -0.9500589 1.5721178 -0.9003176 -0.98809636 -0.01960391]
 pred : [8 4 4 7 6 2 8 1 1 5]
 true : [8 4 3 7 5 1 9 2 0 6]
- 10) 50 testcase
 x : [-0.01717724 0.5732906 -0.661324 0.7665346 -0.28020385 1.383842 -0.04401669 -1.7026945 1.1637585 -1.1820092]
 pred : [4 7 3 7 4 9 5 0 8 1]
 true : [5 6 2 7 3 9 4 0 8 1]

Well we see that the model seemed to have learnt as it has predicted some of the rank correctly. Which means that model has some how encode the arrangements in the train data in the weights. but well it is easy to catch that it doesn't do well with close numbers. For number which are close it repeats itself.

This is a problem as we should atleast have a valid one (may wrong) rather than a invalid one.

Moreover it is unable scale too as it is designed for 10 numbers only.

Listing all the drawbacks:

- invalid seq output
- repeating the ranks for the values nearly same values
- not architecture is not very scale very well as the input size is fixed at 10
- doesn't catch new patterns easily as it is not able to distinguish near values.

Good aspects:

- can easily computed as some matrix multiplication

B. RNN

Well, RNN architecture is a vanilla RNN with a final predicting layer.

Here the input layer will be provided with the ten numbers whose rank to be predicted. And after that it predict 10 values for each input feature and then those 100 values are passed in output layer, which is then softmaxed to get the probabilities of each rank for the given number.

The output Layer helps RNN here to see the entire seq.

Note: normal RNN will be very bad at predicting here , so this modification is added.

1) *Normalisation and Data preparation:* The input are normalised z-score of the original numbers. The output is directly feed into the loss function as nn.CrossEntropy can accept classes as input.

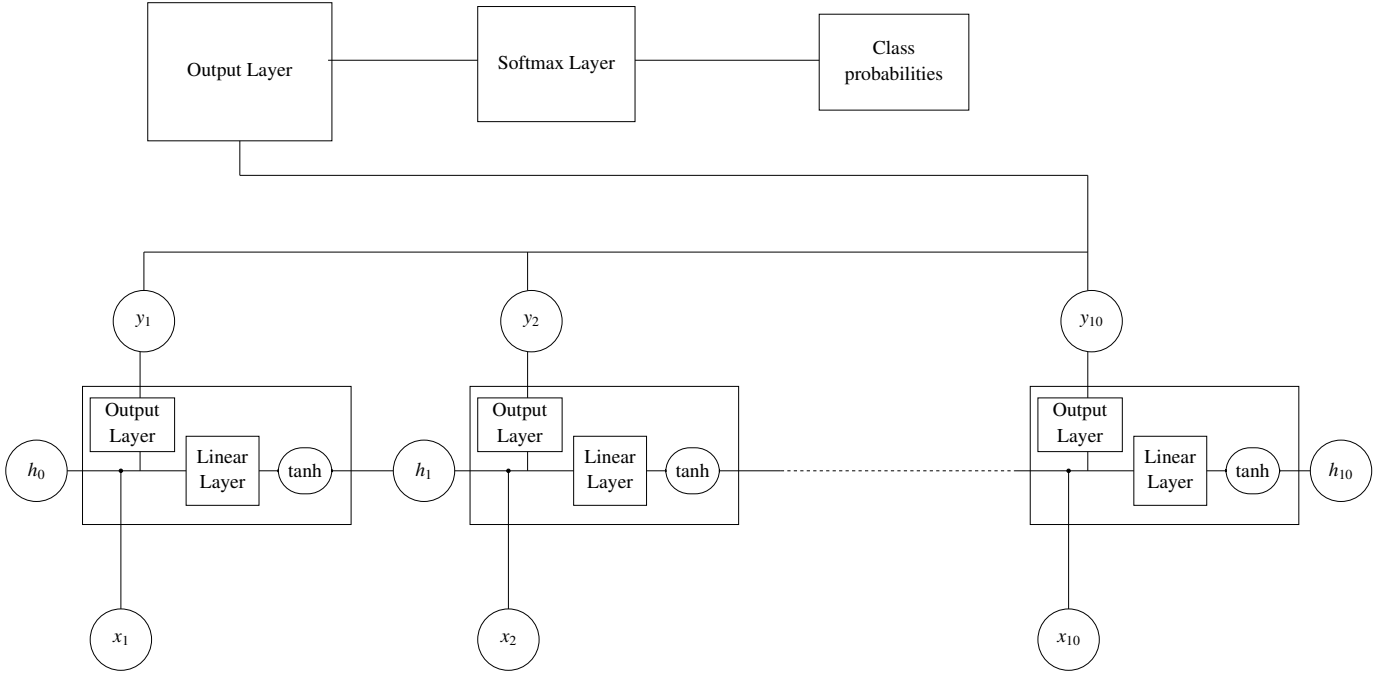


Fig. 3: RNN Architecture

2) *Training:* The training dataset, validation dataset and test dataset are divided in ratio of 8:1:1. The training dataset then divided into batches of size 64. The optimiser used is *torch.optim.AdamW*(*learning_rate* = 0.002) and it is trained for 100 epochs.

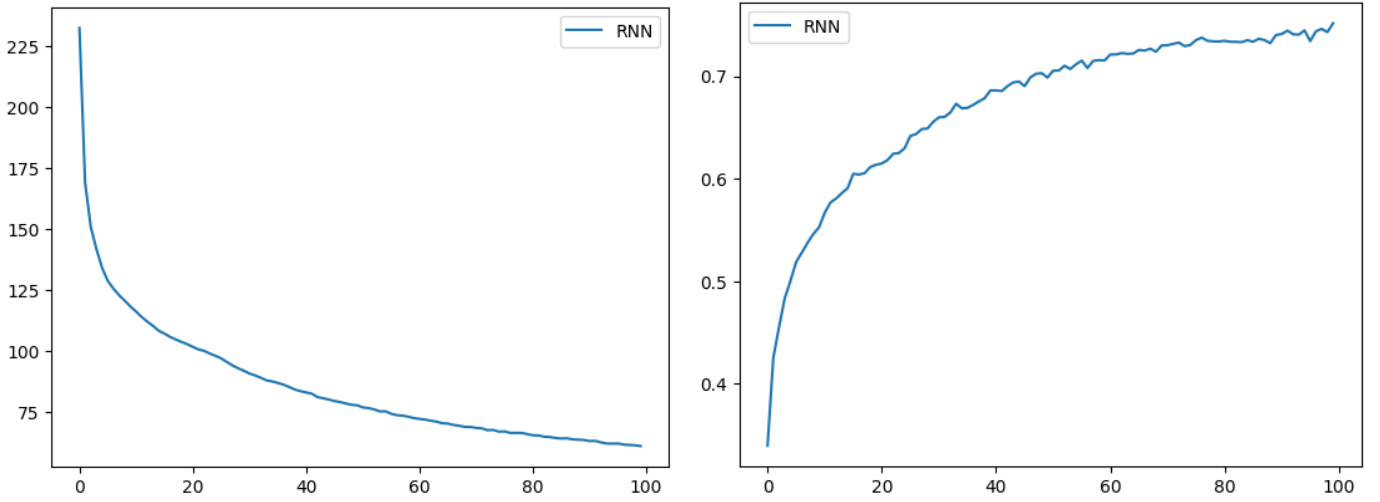


Fig. 4: (Left)loss over the epochs (Right) validation accuracy over epochs

As you can see, it saturates over 75.1% validation accuracy. Well, there are few more metrics apart from this.

3) *Evaluation:*

- 1) Test Accuracy (%): 75.4
- 2) Full sequence correct (%): 7.3
- 3) Atmost 1 incorrect sequence correct (%): 27.2
- 4) Atmost 2 incorrect sequence correct (%): 53.1
- 5) Average no of rank predicted incorrect in a sequence : 2.46

A few testcases are here:

- 1) 5 testcase
x : [[-1.5956196 0.23901713 -0.9372079 -0.24450393 0.19443718 1.7204434 0.9042873 -0.2787962
-0.8857695 0.8837119]]
pred : [0 6 1 3 6 9 7 4 2 7]
true : tensor([0, 6, 1, 4, 5, 9, 8, 3, 2, 7])
- 2) 10 testcase
x : [[1.0385584 0.26576534 0.59875745 1.1202357 -0.5353005 0.22178528 -1.1227487 -1.1290315
1.0479827 -1.5060037]]
pred : [8 5 6 8 3 5 1 1 8 0]
true : tensor([7, 5, 6, 9, 3, 4, 2, 1, 8, 0])
- 3) 15 testcase
x : [[0.5755595 -1.6082455 -0.9784825 0.56616 -0.90015376 1.321249 1.3118496 0.27164406 -
0.66516757 0.10558712]]
pred : [7 0 1 6 2 9 9 4 3 5]
true : tensor([7, 0, 1, 6, 2, 9, 8, 5, 3, 4])
- 4) 20 testcase
x : [[-0.6699673 0.6933111 -0.96876806 -0.39917913 -0.5299044 -1.5710382 0.34782276 0.5252357
0.7773488 1.7951388]]
pred : [2 7 1 3 3 0 5 6 7 9]
true : tensor([2, 7, 1, 4, 3, 0, 5, 6, 8, 9])
- 5) 25 testcase
x : [[-0.732511 1.1469384 0.41175953 -0.9963454 -1.0437763 0.53033674 1.2151204 1.2447647
-0.96373665 -0.81255066]]
pred : [3 8 6 3 1 6 8 8 3 3]
true : tensor([4, 7, 5, 1, 0, 6, 8, 9, 2, 3])
- 6) 30 testcase
x : [[-1.3681347 -0.73531073 0.8606407 1.4564214 -0.6149198 -1.0069621 -0.4111813 1.187857
0.842119 -0.21052976]]
pred : [0 2 7 9 3 1 4 8 6 5]
true : tensor([0, 2, 7, 9, 3, 1, 4, 8, 6, 5])
- 7) 35 testcase
x : [[0.48758876 1.5335821 0.17898828 0.5687995 -0.82152694 1.1242803 -0.8605081 0.3284159
-1.4159889 -1.1236306]]
pred : [6 9 4 7 2 8 1 5 0 1]
true : tensor([6, 9, 4, 7, 3, 8, 2, 5, 0, 1])
- 8) 40 testcase
x : [[0.48174942 -0.9599824 -1.8127141 0.1535503 0.41142103 0.3498837 0.63412756 -1.3145547
1.0414461 1.015073]]
pred : [6 2 0 4 7 4 8 1 8 9]
true : tensor([6, 2, 0, 3, 5, 4, 7, 1, 9, 8])
- 9) 45 testcase
x : [[-0.30514166 -1.0679958 0.43754897 -0.9873416 0.80721396 -0.02621259 1.7649825 -1.0612745
-0.6781672 1.1163883]]
pred : [4 0 6 2 7 5 9 0 2 8]
true : tensor([4, 0, 6, 2, 7, 5, 9, 1, 3, 8])
- 10) 50 testcase
x : [[-0.60002077 -0.8662416 -0.6546302 0.1679239 -0.09829692 1.4034616 -0.18021101 1.9051853
-1.316769 0.23959872]]

pred : [3 1 3 6 6 8 4 9 0 7]

true : tensor([3, 1, 2, 6, 5, 8, 4, 9, 0, 7])

Well we see that the model seemed to have learnt as it has predicted some of the rank correctly. The model is able to predict very well. but well it is easy to catch that is doesn't do well with numbers with lesser difference. For number which are close it repeats itself.

Moreover, it also swap places for separated too far.

Moreover it is unable scale too as it is designed for 10 numbers only.

Listing all the drawbacks:

- invalid seq output but better than previous.
- repeating the ranks for the values nearly same values
- not architecture is not very scale very well as the input size is fixed at 10
- doesn't catch new patterns easily as it is not able to distinguish near values.
- forget far values
- computational heavy

Good aspects:

- better accuracy and better understanding of the problem

C. LSTM

Well, LSTM architecture is a simple implementation.

Here the input layer will be provided with the ten numbers whose rank to be predicted. And after that it iterates over each value and stores those values in history and cell state. After that these vector are passed into a output layer and then softmax is applied.

The output layer helps here to rank the entire seq.

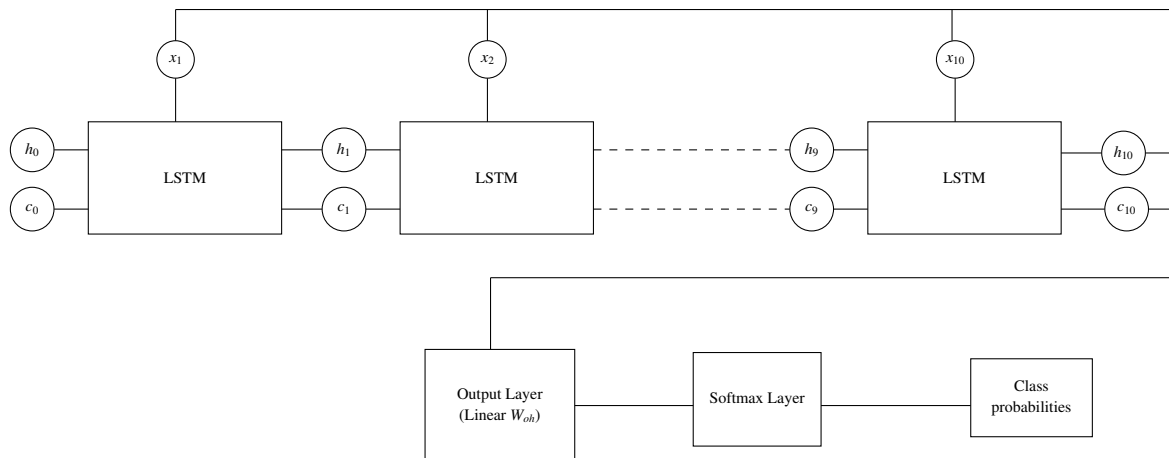


Fig. 5: LSTM Architecture

1) *Normalisation and Data preparation:* The input are normalised z-score of the original numbers. The output is directly feed into the loss function as nn.CrossEntropy can accept classes as input.

2) *Training:* The training dataset, validation dataset and test dataset are divided in ratio of 8:1:1. The training dataset then divided into batches of size 64. The optimiser used is *torch.optim.AdamW*(*learning_rate* = 0.001) and it is trained for 100 epochs.

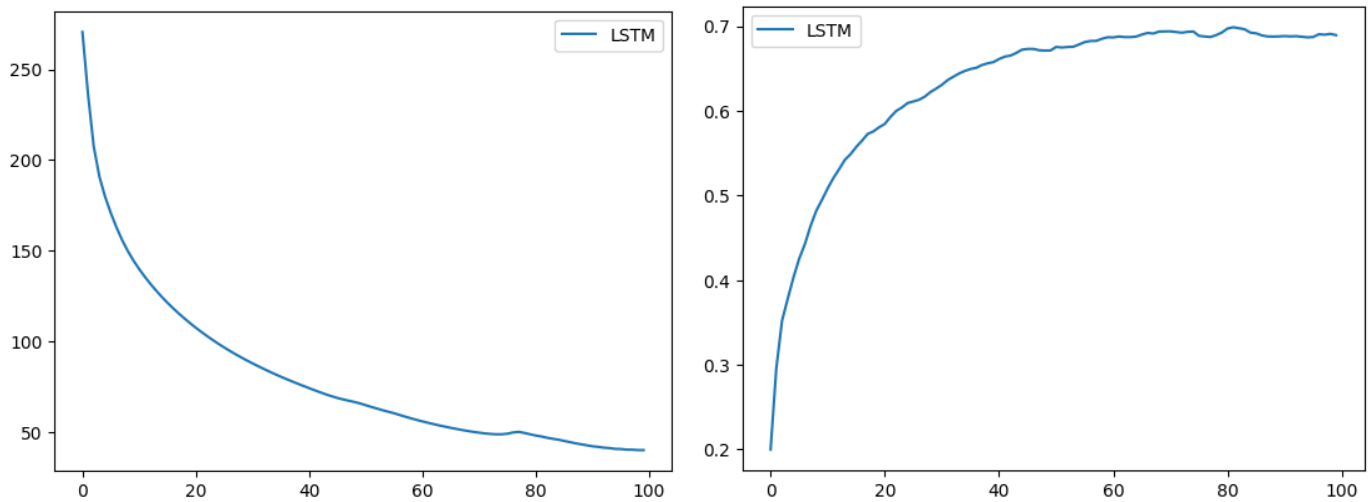


Fig. 6: (Left)loss over the epochs (Right) validation accuracy over epochs

As you can see, it saturates over 68.8% validation accuracy. Well, there are few more metrics apart from this.

3) Evaluation:

- 1) Test Accuracy (%): 68.81
- 2) Full sequence correct (%): 4.3
- 3) Atmost 1 incorrect sequence correct (%): 16.1
- 4) Atmost 2 incorrect sequence correct (%): 36.3
- 5) Average no of rank predicted incorrect in a sequence : 3.119

A few testcases are here:

- 1) 5 testcase
 x : [[0.6226948 1.2467268 -1.2360291 -0.3133532 -1.151339 0.9926566 -0.2999811 0.28393456
 1.1397498 -1.2850602]]
 pred : [6 9 1 4 2 7 4 6 8 1]
 true : tensor([6, 9, 1, 3, 2, 7, 4, 5, 8, 0])
- 2) 10 testcase
 x : [[-0.6788334 -0.05816858 -0.00419773 -1.5153817 1.1711676 -1.6053331 0.44256103 1.2641174
 0.4125772 0.57149136]]
 pred : [2 3 3 1 8 0 4 9 6 7]
 true : tensor([2, 3, 4, 1, 8, 0, 6, 9, 5, 7])
- 3) 15 testcase
 x : [[-0.68435204 0.42710373 0.9828316 0.86630803 -1.8719964 1.2427689 -0.19584931 0.6063708
 -1.0294411 -0.34374464]]
 pred : [2 5 7 7 0 9 4 7 1 4]
 true : tensor([2, 5, 8, 7, 0, 9, 4, 6, 1, 3])
- 4) 20 testcase
 x : [[-0.41699928 0.06130363 0.6541298 -1.9832729 -0.21153113 1.2705343 -0.8548822 -0.45068258
 0.9269645 1.0044361]]
 pred : [2 5 6 0 4 9 1 3 8 7]
 true : tensor([3, 5, 6, 0, 4, 9, 1, 2, 7, 8])
- 5) 25 testcase
 x : [[-0.8204587 0.14929734 0.06359797 -0.621997 -1.4068229 1.7234596 0.05457698 1.6061867
 -0.14839523 -0.59944457]]
 pred : [1 7 5 2 0 8 6 9 5 2]

```

true : tensor([1, 7, 6, 2, 0, 9, 5, 8, 4, 3])
6) 30 testcase
x : [[-0.0518996 -0.7828802 1.5197088 -0.46855855 -0.96562535 -0.63668406 1.9656069 0.08698671
-0.79384494 0.12719063]]
pred : [5 2 9 4 0 3 9 7 1 6]
true : tensor([5, 2, 8, 4, 0, 3, 9, 6, 1, 7])
7) 35 testcase
x : [[ 0.5403877 0.02626885 1.2458938 -1.0882808 -0.5441404 -0.439065 -1.2158724 -0.9344204
0.86687195 1.5423566 ]]
pred : [6 5 8 1 3 4 1 2 8 9]
true : tensor([6, 5, 8, 1, 3, 4, 0, 2, 7, 9])
8) 40 testcase
x : [[ 0.5949918 -0.5577821 1.379316 -1.0976888 -0.550486 -0.52859795 0.6387681 0.9926259
-1.6339476 0.7628007 ]]
pred : [6 2 9 1 3 4 6 8 0 6]
true : tensor([5, 2, 9, 1, 3, 4, 6, 8, 0, 7])
9) 45 testcase
x : [[-0.2971111 -1.9077842 0.17868957 0.3020453 0.7672726 0.24917856 1.4510158 0.93644613
-1.0689654 -0.6107871 ]]
pred : [3 0 4 6 8 5 8 8 1 2]
true : tensor([3, 0, 4, 6, 7, 5, 9, 8, 1, 2])
10) 50 testcase
x : [[-0.5694185 0.02958956 0.10897616 -1.464322 1.4657654 1.4765909 0.48786676 -0.15444301
-1.3885438 0.00793867]]
pred : [2 5 5 0 9 9 6 4 1 5]
true : tensor([2, 5, 6, 0, 8, 9, 7, 3, 1, 4])

```

Well we see that the model seemed to have learnt as it has predicted some of the rank correctly. The model is able to predict very well. but well it is easy to catch that it doesn't do well with numbers with lesser difference. For number which are close it repeats itself.

Listing all the drawbacks:

- invalid seq output but better than previous.
- repeating the ranks for the values nearly same values
- forget far values
- computational heavy

Good aspects:

- better accuracy and better understanding of the problem
- better memory
- can scaled for bigger sequences

III. IMPLEMENTING ENCODER-ONLY TRANSFORMER

Well, This is the best of all the model - Encoder-Only Transformer.

The first implementation will be very identical to the paper but with slight variation.

The output layer helps here to rank the entire seq.

A. Architecture

1) *Normalisation and Data preparation:* The input are normalised z-score of the original numbers. In the embedding, the normalised z-score is provided itself. The output is directly feed into the loss function as nn.CrossEntropy can accept classes as input.

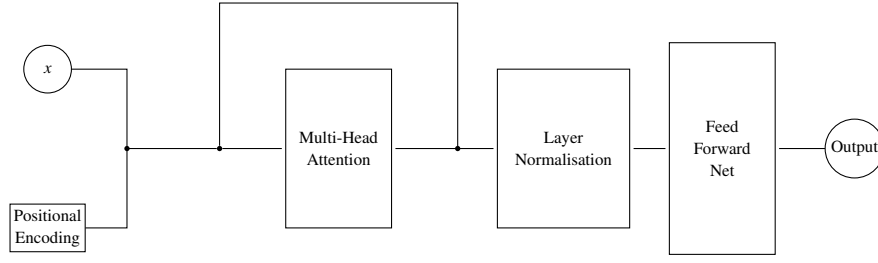


Fig. 7: Transformer Architecture

2) *Positional Encoding*: Well, as the embedding dimension is one, the positional encoding suggested by the paper will not work here. So, I used another system of encoding:

$$P_{pos} = (1.5)^{-pos}$$

Note: any constant greater than 1 works.

It has great properties: it is unique for every position; stays in (0,1]; and $1.5^{(-9)}$ is 0.026 which not negligible; can easily understood by the transformer.

B. Training

The training dataset, validation dataset and test dataset are divided in ratio of 8:1:1. The training dataset then divided into batches of size 64. The optimiser used is `torch.optim.AdamW(learning_rate = 0.001)` and it is trained for 200 epochs.

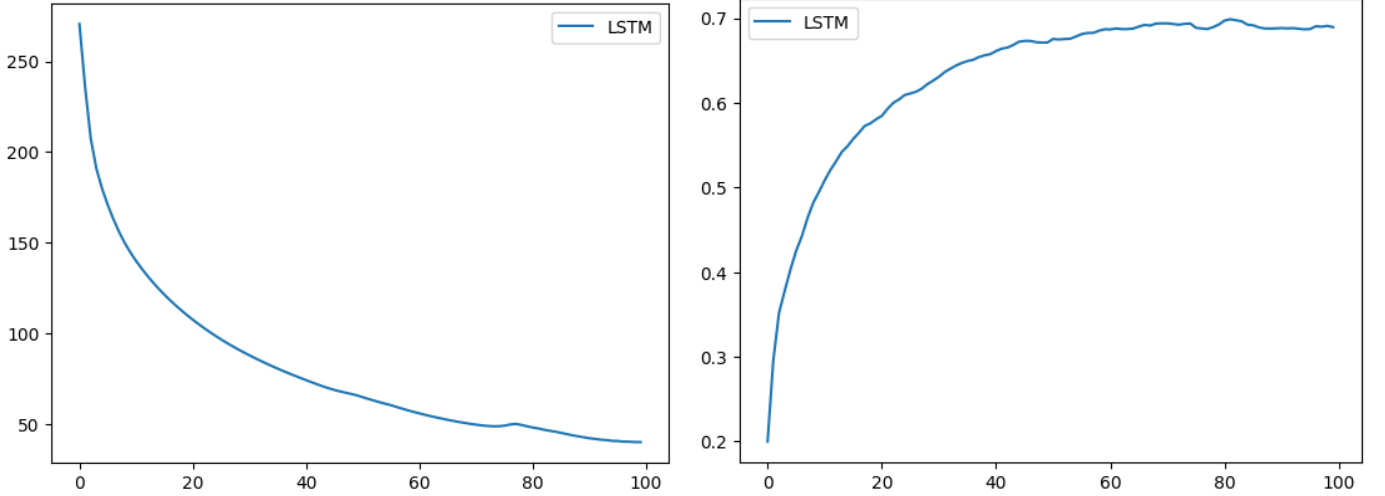


Fig. 8: (Left)loss over the epochs (Right) validation accuracy over epochs

As you can see, it saturates over 74.8% validation accuracy. Well, there are few more metrics apart from this.

1) Evaluation:

- 1) Test Accuracy (%): 73.79
- 2) Full sequence correct (%): 6
- 3) Atmost 1 incorrect sequence correct (%): 20.3
- 4) Atmost 2 incorrect sequence correct (%): 46.5
- 5) Average no of rank predicted incorrect in a sequence : 2.621

A few testcases are here:

- 1) 5 testcase
x : -1.0139829 0.9693663 -1.2405518 -0.87804157 1.7884998 0.54411393 -0.9721548 0.33497342
0.05960507 0.4081726
pred : 2 8 0 2 9 7 2 6 5 6
true : 1, 8, 0, 3, 9, 7, 2, 5, 4, 6
- 2) 10 testcase
x : [[0.50953513 0.2765529 0.79395497 0.82118666 0.4883549 0.87565 -1.8414671 -0.3285957
0.09198257 -1.6871543]]
pred : [5 4 8 8 5 9 0 2 4 1]
true : tensor([6, 4, 7, 8, 5, 9, 0, 2, 3, 1])
- 3) 15 testcase
x : [[-0.9339631 -0.78526545 1.1655067 -0.20463634 -0.08426202 1.0132686 -1.479188 1.0876174
1.0274303 -0.80650795]]
pred : [2 2 9 4 4 7 0 8 7 2]
true : tensor([1, 3, 9, 4, 5, 6, 0, 8, 7, 2])
- 4) 20 testcase
x : [[1.863396 0.3594803 -1.1000161 -1.0080044 0.64503396 -0.94454795 0.9876983 -0.11326961
-0.9096469 0.21987629]]
pred : [9 6 0 1 7 2 8 5 2 6]
true : tensor([9, 6, 0, 1, 7, 2, 8, 4, 3, 5])
- 5) 25 testcase
x : [[-0.66240436 0.81871426 0.7476643 -1.3783693 -1.6571039 0.73126817 -0.74438506 0.5181182
0.8679027 0.75859505]]
pred : [2 8 6 1 0 6 2 5 9 7]
true : tensor([3, 8, 6, 1, 0, 5, 2, 4, 9, 7])
- 6) 30 testcase
x : [[0.19609028 -1.9665515 -0.29615283 0.16381204 -0.19528334 -0.46964836 0.5592205 0.769029
1.8543847 -0.61490035]]
pred : [5 0 3 5 3 2 7 8 9 1]
true : tensor([6, 0, 3, 5, 4, 2, 7, 8, 9, 1])
- 7) 35 testcase
x : [[-1.5127878 -0.39488912 2.0393066 -0.11636829 -0.20030607 -0.23845962 -0.25372103 -0.41778123
-0.30332065 1.3983272]]
pred : [0 3 9 5 4 4 4 3 4 8]
true : tensor([0, 2, 9, 7, 6, 5, 4, 1, 3, 8])
- 8) 40 testcase
x : [[0.08240458 -1.2603298 1.2494543 1.6050069 0.01965998 -1.0930109 -0.8211177 0.62200814
-0.8838623 0.47978705]]
pred : [5 0 8 9 5 1 2 6 2 6]
true : tensor([5, 0, 8, 9, 4, 1, 3, 7, 2, 6])
- 9) 45 testcase
x : [[-1.1409974 1.6209114 -1.004196 -1.0633534 0.7372485 0.84447116 0.20113498 0.9221152
-0.84151334 -0.27582115]]
pred : [0 9 2 1 7 7 5 7 2 4]
true : tensor([0, 9, 2, 1, 6, 7, 5, 8, 3, 4])
- 10) 50 testcase
x : [[1.2786505 0.5309186 -0.14752549 1.1429617 -0.39292017 0.41255176 -1.1435392 -1.3802729
0.89179313 -1.1926181]]
pred : [9 6 4 8 3 5 1 0 7 1]
true : tensor([9, 6, 4, 8, 3, 5, 2, 0, 7, 1])

Well we see that the model seemed to have learnt as it has predicted some of the rank correctly. The model is able to predict very well. but well it is easy to catch that it doesn't do well with numbers with lesser difference. For number which are close it repeats itself.

Listing all the drawbacks:

- invalid seq output but better than previous.
- repeating the ranks for the values nearly same values

Good aspects:

- computational friendly
- better accuracy and better understanding of the problem
- self-attention is much better than the other approaches
- can scaled for bigger sequences by increasing output tokens
- the difference in the values where it predicts the same value is order of 10^{-2} which is smaller than others.

IV. EXPERIMENTS AND ABLATION STUDIES

These are included in the inferenceAndExperiment.ipynb.

V. SOURCE CODES

Each model can be found in its respective named directory and the notebooks are in the main directory only.

../models and training.ipynb : the testing and running different models

Source code for the report is in the "report" folder.